Filter Engine Reference Manual

for Ezairo® 7100

M-20788-011
July 2014

© SCILLC, 2014
Previous Edition © 2013
"All Rights Reserved"

ON Semiconductor®

ON Semiconductor and ON are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is either a trademark or registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

Table of Contents

Page

1. Introduction . . . 5
   - 1.1 Purpose . . . 5
   - 1.2 Intended Audience . . . 5
   - 1.3 Conventions . . . 5
   - 1.4 Manual Organization . . . 5
   - 1.5 Further Reading . . . 5

2. Processor Overview . . . 7

3. Filter Engine Core . . . 9
   - 3.1 Infinite Impulse Response (IIR) Filters . . . 9
   - 3.2 Finite Impulse Response (FIR) Filters . . . 9
   - 3.3 Coefficient Storage . . . 10
   - 3.4 Filter State Variables . . . 10
   - 3.5 Dynamic Coefficient Update . . . 10
     - 3.5.1 Active and Shadow Registers for Dynamic Coefficient Updates . . . 11

4. Register File . . . 12
   - 4.1 Register File Groups . . . 12
     - 4.1.1 Extended-Precision Data Registers . . . 12
     - 4.1.2 Single-Precision Data Registers . . . 12
     - 4.1.3 Input Data Registers . . . 12
     - 4.1.4 Output Data Registers . . . 13
     - 4.1.5 Register Summary . . . 13
   - 4.2 Input and Output Shadow Registers . . . 14

5. Program Execution and Instruction Set . . . 15
   - 5.1 Starting Program Execution . . . 15
   - 5.2 Conditional Processing . . . 15
     - 5.2.1 Active and Shadow Registers for Conditional Processing . . . 15
   - 5.3 Multi-Rate Processing . . . 16
     - 5.3.1 Multi-Rate Counters . . . 16
     - 5.3.2 Phase Memory . . . 16
   - 5.4 Numerical Precision and Data Conversion . . . 17
   - 5.5 Program Format and Instruction Set . . . 17
     - 5.5.1 Program Format . . . 17
     - 5.5.2 General Instruction Format . . . 18
     - 5.5.3 Instruction Descriptions . . . 18
   - ABS . . . 20
   - ADD . . . 21
   - BIQ . . . 22
   - CMP . . . 23
   - FIR . . . 25
   - LDET . . . 27
   - LMLA . . . 29
   - LMLS . . . 30
   - LMUL . . . 31
   - LOG . . . 32

MLA 33
MLS 34
MOV 35
MUL 36
NOP 37
SUB 38
6. Software Tools 39
6.1 Filter Engine Assembler Usage 39
6.2 Filter Engine Control and Configuration Through the CFX 40
6.3 The Preprocessor 40
6.4 Assembler Operators 41
6.4.1 Arithmetic and Logical Operators 41
6.4.2 Rounding Operators 41
6.4.3 Special Operators 42
6.4.4 Comments and Labels 42
6.5 Debugging Filter Engine Configurations 42

CHAPTER 1
Introduction

1.1 Purpose

The *Filter Engine Reference Manual* provides information about programming the Filter Engine, the Filter Engine assembly language, execution of Filter Engine instructions, and the software tools that support the use of this core.

1.2 Intended Audience

This manual is for anyone who wants to program the Filter Engine. It is also useful for those who are writing CFX applications that interface to the Filter Engine. For information about CFX control and configuration of Filter Engine program execution, refer to the *Hardware Reference Manual for Ezairo 7100*. For information about macros supporting CFX control and configuration of the Filter Engine, refer to the *Firmware Reference Manual for Ezairo 7100*.

This manual assumes that you have a general familiarity with the system architecture of the Ezairo® 7100 chip, and you are familiar with digital signal processing (DSP), including the underlying theory and use of real-time DSP applications.

1.3 Conventions

The following conventions are used in this manual to signify particular types of information:

- Control and configuration registers are shown in a `monospace font`.
- **`monospace bold`** identifies an optional parameter or indicates a placeholder for specific information.
- In general, numbers are presented in decimal notation. In cases where hexadecimal or binary notation is more convenient, these numbers are identified by the prefixes “0x” and “0b” respectively. For example, the decimal number 123456 can also be represented as 0x1E240 or 0b11110001001000000.

1.4 Manual Organization

The *Filter Engine Reference Manual* contains the following chapters:

- **Chapter 1: Introduction**, introduces this manual—explaining the purpose, intended audience, conventions used and organization. It also contains a list of further reading materials that fill in the gaps when developing applications for the Filter Engine.
- **Chapter 2: Processor Overview**, provides a high-level description of the core components.
- **Chapter 3: Filter Engine Core**, describes the filtering available on this core.
- **Chapter 4: Register File**, describes the registers available for storing input, output, or intermediate data.
- **Chapter 5: Program Execution and Instruction Set**, explains how the program memory works, provides general information about the format of programs, and describes each instruction in a reference style.
- **Chapter 6: Software Tools**, explains how to use the toolchain to preprocess, assemble and compile an application, and how to debug it.

1.5 Further Reading

For more information, refer to the following documents:

- *Getting Started Guide for Ezairo 7100*
- *Hardware Reference Manual for Ezairo 7100*
- *Firmware Reference Manual for Ezairo 7100*

- *SK4 ELF Toolchain Reference for Ezairo 7100*
- *CFX DSP Architecture Manual*
- *HEAR Configurable Accelerator Reference Manual*

In particular, the Filter Engine chapter and the input/output audio control chapter in the *Hardware Reference Manual for Ezairo 7100* are especially relevant. Together with this manual, they provide enough information for you to configure the Filter Engine from the CFX, configure the audio path with the Filter Engine in it, and use the Filter Engine in the Ezairo 7100 system.

CHAPTER 2
Processor Overview

The Filter Engine is a core that provides a low group delay processing path and basic filtering capabilities for the Ezairo 7100 system.

It can implement filters (either FIR or IIR) with a total of up to 160 coefficients. FIR filters are implemented using a direct-form structure. IIR filters are implemented with a cascade of second-order sections (biquads), each implemented as a direct-form I filter.

The Filter Engine consists of the following components, as shown in Figure 1 on page 7:

- A core block, which is designed to efficiently implement filtering operations such as Biquad IIR and FIR
- An Arithmetic Logic Unit (ALU) to perform a total of 13 different arithmetic instructions, such as addition, subtraction, multiplication and shift
- A multi-port register file to store intermediate values between the core stages as well as the ALU data
- A program memory and instruction set to control the operation of the Filter Engine
- A phase memory to support multi-rate processing
- A decoder that fetches and decodes instructions from the program memory. It also utilizes multi-rate counters to decode the phase memory content to control the program memory address pointer increments
- An interface for communication with the CFX

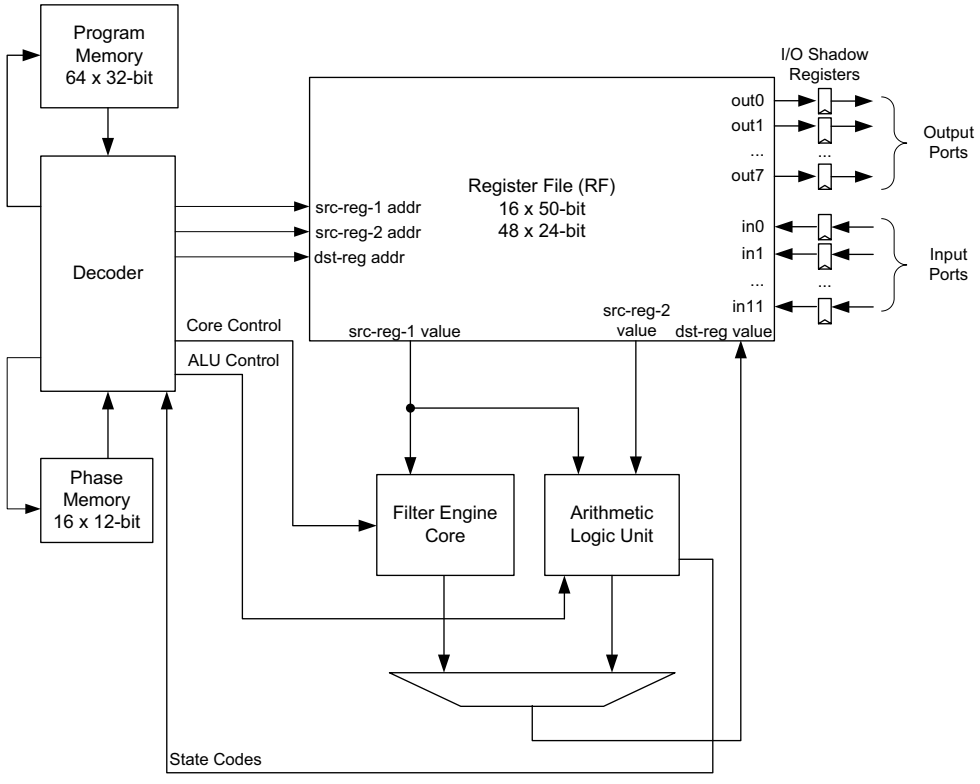


Figure 1. Filter Engine Functional Block Diagram

The Filter Engine is programmable, but does not include direct debugging access. The CFX controls the Filter Engine behavior and can monitor the Filter Engine state through control and configuration registers. To understand how to configure the Filter Engine from the CFX and to use the Filter Engine in the Ezairo 7100 system, see the chapter about the Filter Engine in the *Hardware Reference Manual for Ezairo 7100* in conjunction with this manual.

In addition, the CFX defines how the Filter Engine fits in the audio path of the Ezairo 7100 system. See the Filter Engine chapter and the input/output audio control chapter of the *Hardware Reference Manual for Ezairo 7100*.

CHAPTER 3
Filter Engine Core

The core block of the Filter Engine is designed to efficiently implement Biquad-IIR and FIR filtering operations. It can implement up to 32 stages of IIR and FIR in series or in parallel, where each stage consists of either one biquad section or 5-tap FIR filter with accumulation. Therefore the Filter Engine core can implement an IIR filter up to the 64th-order, or an FIR filter of up to 160 taps. A set of coefficients, `C`**`x`** (where **`x`** = 0, 1, 2, ..., 31), and a set of state variables `SV`**`y`** (where **`y`** = 0, 1, 2, ..., 31), are required to accompany each stage. The coefficients can be static, or dynamic (under CFX control).

3.1 Infinite Impulse Response (IIR) Filters

IIR filters are programmed as second-order sections (biquads). All the operations required for a single biquad are executed using the `BIQ` instruction. The inputs and outputs for biquad sections are stored in the register file. Quantization is applied to the output of each `BIQ` instruction before the result is stored in the destination register. Cascade or parallel connections of biquad sections can be implemented using the appropriate source and destination registers in the instruction.

Quantization Management

If the destination register is 24 bits, quantization has two modes, which are controlled by the QF field of the opcode instruction:

- QF = 0: round
- QF = 1: error spectral shaping

  The error spectral shaping mode implements a quantization noise shaping function, which can be useful in certain IIR filters. The algorithm is:

$$y(i) = a(i) + 2 \bullet e(i-1) + e(i-2)$$

  where:

  - y = quantized signal
  - a = accumulator output
  - e = 22 LSBs of a
  - i = sample index

3.2 Finite Impulse Response (FIR) Filters

FIR filters are programmed in five-tap sections. For FIR filters longer than five taps, you need to use multiple `FIR` instructions. Put these `FIR` instructions in the same order as the taps in the transfer function. The first `FIR` instruction implements the first 5 taps of the filter, the second `FIR` instruction implements the next 5 taps of the filter, and so on. The Filter Engine core has an internal accumulator that keeps the partial filtering result for multi-stage FIR filters. Use of the accumulator depends on the data width of the input register specified for the FIR instruction.

If the input is single-precision, the accumulator is cleared for each opcode instruction. Extended-precision registers (e0 - e15) must be used for all intermediate and final FIR filter stages so that the accumulator and internal registers retain the temporary output of the previous stage. For multi-stage FIR filters, the associated state variable sets must be consecutive (see Section 3.4, “Filter State Variables” on page 10).

3.3 Coefficient Storage

Up to 32 sets of biquad IIR or 5-tap FIR coefficients can be stored at address locations 0xF900-0xF99F in PMEM. Each set of coefficients (C0 to C31) occupies five words. Within each section, the coefficients are arranged as follows:

- Biquad IIR: the coefficients are in m2p22 format, and organized as b_0, b_1, b_2, $-a_1$, $-a_2$ for the transfer function

$$H(z) = \frac{b_0 + b_1 \cdot z^{-1} + b_2 \cdot z^{-2}}{1 + a_1 \cdot z^{-2} + a_2 \cdot z^{-2}}$$

- FIR: the coefficients are in frac24 format, and organized as (b_0, b_1, b_2, b_3, b_4), (b_5, b_6, b_7, b_8, b_9), ..., for the transfer function

$$H(z) = \sum_{k=0}^{N} b_k \cdot z^{-k}$$

3.4 Filter State Variables

For each stage of a biquad IIR or FIR filter, you need both a coefficient set and a state variable set. The state variable set keeps the intermediate filtering results for the associated stage. The index of the state variable set does not need to match the index of the coefficient set.

Up to 32 sets of state variables can be stored at address locations 0xF800-0xF8BF in PMEM: each set of state variables (SV0 to SV31) occupies six words. For a multi-stage FIR filter, assign the state variable set for each consecutive FIR filter stage so that they are consecutive in memory. For a multi-stage biquad IIR filter, the state variable set assignment has no restrictions.

3.5 Dynamic Coefficient Update

You can switch between different coefficient sets when the Filter Engine is active. This allows dynamic coefficient update by writing a new set of coefficients to a different location in the coefficient memory, and switching to the new set in the next program cycle.

Dynamic coefficient update is supported through the use of a coefficient set offset value. The coefficient offset, which is set in bit field COEFF_OFFSET in the D_FENG_CFG register, specifies an address offset for the Filter Engine to use when addressing the coefficient memory. The use of the offset is controlled on a per-coefficient set basis through the D_FENG_COEFF_OFFSET_EN_ACTIVE register (which is mirrored in the D_FENG_COEFF_OFFSET_EN_SHADOW register as described in Section 3.5.1, “Active and Shadow Registers for Dynamic Coefficient Updates” on page 11). Setting a bit in this register enables the use of the offset for the corresponding coefficient set, while clearing a bit disables the use of the offset. Thus, coefficient set C**n** is used when bit **n** is cleared in the D_FENG_COEFF_OFFSET_EN_SHADOW register, and coefficient set C**m** (where **m** = **n** + the value of COEFF_OFFSET) is used when bit **n** is set in the D_FENG_COEFF_OFFSET_EN_SHADOW register. Changes to the control bits in the D_FENG_COEFF_OFFSET_EN_SHADOW register take effect in the following program execution cycle.

For example, if the COEFF_OFFSET bit field in the D_FENG_CFG register is set to 2, the D_FENG_COEFF_OFFSET_EN_SHADOW register contains all zeros and the following snippet is executed:

```
biq s0, in8, c0, sv0, rnd
fir out6, s0, c1, sv1
```

The biq instruction uses coefficient set c0 and the fir instruction will use coefficient set c1.

If bit 0 of the D_FENG_COEFF_OFFSET_EN_SHADOW register is now set, in the next program execution cycle the biq instruction will use coefficient set c2 (because the offset enable for coefficient set c0 is enabled with an offset value of 2), while the fir instruction will continue to use coefficient set c1.

Now if bits 0 and 1 of the D_FENG_COEFF_OFFSET_EN_SHADOW register are set, in the next program execution cycle the biq instruction will use coefficient set c2 (c0 plus the offset 2), and the fir instruction will use coefficient set c3 (c1 plus the offset 2).

The current coefficient offset enable status can be read through the D_FENG_COEFF_OFFSET_EN_ACTIVE register.

3.5.1 Active and Shadow Registers for Dynamic Coefficient Updates

The Filter Engine coefficient offset enable is determined by the value of the configuration register D_FENG_COEFF_OFFSET_EN_ACTIVE. It has a shadow register, D_FENG_COEFF_OFFSET_EN_SHADOW, which the CFX can use to safely write a new value for the coefficient offset enable. When the CFX writes to the shadow register, its content is transferred to the active register as soon as the Filter Engine becomes idle. If the Filter Engine is continuously active, the transfer occurs at the start of the next program execution.

The CFX can also write to the active register to make the new value take effect immediately. However, use this feature with caution to avoid unintended memory access conflicts. If a memory conflict occurs, the Filter Engine access to the active register takes precedence over the CFX's access. Any data read by the CFX is invalid and any data written by the CFX is lost.

CHAPTER 4
Register File

There are 64 register locations with 24-bit or 50-bit word lengths to store input, output and intermediate data. Reading and writing to the same address under the same instruction is permitted. In this case, the read occurs first; the read data is processed, and at the end of the instruction execution, the result is written to the same address location.

All register file locations can be accessed by both the Filter Engine and the CFX. If the Filter Engine and CFX attempt to write access the same register file location conflicts, only the Filter Engine write access succeeds. The CFX is notified about the conflict through the `REG_ERR` bit field of the `D_FENG_INT_STATUS` register. The data read by the CFX is invalid, and the data to be written by the CFX is lost.

4.1 Register File Groups

4.1.1 Extended-Precision Data Registers

Filter Engine instructions use registers `RF_EXTENDED_0` to `RF_EXTENDED_15` to store intermediate data with higher resolution. These registers are not linked to the I/O interface. These 50-bit extended-precision registers have an m4p46 format when used to store linear audio data, or an m11p39 format when used to store log2-format data. In the Filter Engine source code, they appear as `e0` to `e15`. When the CFX accesses them over the 32-bit P memory bus, the register names are `D_FENG_RF_EXTENDED_0` to `D_FENG_RF_EXTENDED_15`, and the 32 MSBs are read or written.

4.1.2 Single-Precision Data Registers

Registers `RF_SINGLE_0` to `RF_SINGLE_27` store intermediate data with lower resolution. These registers are not linked to the I/O interface. These 24-bit single-precision registers have a frac24 format when used to store linear audio data, or an m8p16 format when used to store log2-format data. In the Filter Engine source code, they appear as `s0` to `s27`. When the CFX accesses them, the register names are `D_FENG_RF_SINGLE_0` to `D_FENG_RF_SINGLE_27`.

4.1.3 Input Data Registers

Registers `RF_IN_0` to `RF_IN_11` hold input data and are linked to the input stage, the output stage, and the IOC. You can also use the input data registers to store single-precision intermediate data if you run out of intermediate data registers. These 24-bit input data registers have a frac24 format when used to store linear audio data, or an m8p16 format when used to store log2-format data. In the Filter Engine source code, they appear as `in0` to `in11`. When the CFX accesses them, the register names are `D_FENG_RF_IN_0` to `D_FENG_RF_IN_11`.

Table 1. Input Data Register Sources

Filter Engine Input Data Register	Data Source
in0	ADC0 data, decimated
in1	ADC1 data, decimated
in2	ADC2 data, decimated
in3	ADC3 data, decimated
in4	IOC output 0 (commonly used for DAC data before Filter Engine post-filtering)
in5	IOC output 1 (commonly used for DAC data before Filter Engine post-filtering)
in6	Interpolation filter 0 output (LDP)

Table 1. Input Data Register Sources (Continued)

Filter Engine Input Data Register	Data Source
in7	Interpolation filter 1 output (LDP)
in8	ADC0 data, undecimated (LDP)
in9	ADC1 data, undecimated (LDP)
in10	ADC2 data, undecimated (LDP)
in11	ADC3 data, undecimated (LDP)

4.1.4 Output Data Registers

Registers `RF_OUT_0` to `RF_OUT_7` hold output data and are linked to the IOC and the output stage. You can also use the output data registers to store single-precision intermediate data if you run out of intermediate data registers. These 24-bit output data registers have a frac24 format when used to store linear audio data, or an m8p16 format when used to store log2-format data. In the Filter Engine source code, they appear as `out0` to `out7`. When the CFX accesses them, the register names are `D_FENG_RF_OUT_0` to `D_FENG_RF_OUT_7`.

Table 2. Output Data Register Destinations

Filter Engine Output Data Register	Data Destination
out0	IOC input 0 (commonly used for ADC data after Filter Engine pre-filtering)
out1	IOC input 1 (commonly used for ADC data after Filter Engine pre-filtering)
out2	IOC input 2 (commonly used for ADC data after Filter Engine pre-filtering)
out3	IOC input 3 (commonly used for ADC data after Filter Engine pre-filtering)
out4	Interpolation filter 0 input
out5	Interpolation filter 1 input
out6	Sigma-Delta Modulator 0 (LDP)
out7	Sigma-Delta Modulator 1 (LDP)

4.1.5 Register Summary

Table 3 summarizes the types of registers, their data widths, and their various names.

Table 3. Summary of Registers

Type of Register	Data Width	General Name	Filter Engine Name	CFX Name
Extended precision	50 bits	`RF_EXTENDED_0` to `RF_EXTENDED_15`	`e0` to `e15`	`D_FENG_RF_EXTENDED_0` to `D_FENG_RF_EXTENDED_15`
Single precision	24 bits	`RF_SINGLE_0` to `RF_SINGLE_27`	`s0` to `s27`	`D_FENG_RF_SINGLE_0` to `D_FENG_RF_SINGLE_27`

Table 3. Summary of Registers

Type of Register	Data Width	General Name	Filter Engine Name	CFX Name
Input Data	24 bits	RF_IN_0 to RF_IN_11	in0 to in11	D_FENG_RF_IN_0 to D_FENG_RF_IN_11
Output Data	24 bits	RF_OUT_0 to RF_OUT_7	out0 to out7	D_FENG_RF_OUT_0 to D_FENG_RF_OUT_7

4.2 Input and Output Shadow Registers

Shadow registers at the input ports ensure that all input samples are synchronized. When an input sample is enabled (by setting the corresponding bit in FIN_EBL in the D_FENG_CFG register), it is captured in its shadow register as soon as it arrives. Then all active shadow registers are transferred to the input data registers in parallel at the start of the program execution.

Shadow registers at the output ports can ensure that all or a subset of the output samples are synchronized. When you write to an output via an output data register (by the Filter Engine or the CFX), its update timing is controlled by the OSR_EBL bit field in the D_FENG_CFG register. The eight bits in this bit field control the shadow registers at the eight output ports:

- If the corresponding bit is set, any update to the output data register is transferred to the output port at the start of the program execution via the output shadow register. Therefore, any outputs that have their OSR_EBL bit set to 1 are synchronized.
- If the corresponding bit is cleared, the output shadow register is bypassed and the update to the output data register is transferred to the output port immediately. This option minimizes system delay and is especially useful for applications requiring a low group delay.

IMPORTANT: Use caution if you use unsynchronized output sample update. Since the output sample capture from the output data register can happen during Filter Engine program execution, the following restrictions apply:

- **Do not use this register file location for intermediate data.**
- **Do not update this register file location through conditional processing by multiple instructions.**
- **In multi-rate processing, if the same register file location is updated in multiple phases, ensure that the delay from the start of the program phase to when this register file location is updated is the same in each program phase.**

CHAPTER 5
Program Execution and Instruction Set

The Filter Engine architecture includes a 64-word (32 bits per word) program memory. The Filter Engine program memory is only writable from the CFX DSP.

5.1 Starting Program Execution

When the Filter Engine is running continuously, Filter Engine program execution starts and runs in this sequence:

1. Program execution is triggered by a *sample ready* signal (SR_IN**x**).
2. All Filter Engine shadow registers are copied to their equivalent active registers. The Filter Engine is brought out of the sleep state, executes the instructions starting from the base of the program memory, and continues until all active instructions are executed.
3. The Filter Engine returns to the sleep state and waits for a new event.

The CFX can monitor the address of the program memory decoder at the `D_FENG_PGM_ADR` register, but cannot modify its content.

The sample ready signal (SR_IN**x**, where **x** = 0, 1, ... 11) is set by the `SRI_SEL` bit field of the `D_FENG_CFG` register and corresponds with the 12 input sources in0 to in11. If this bit-field value exceeds 11, Filter Engine processing is not triggered by any input sample. If you have more than one sampling rate in the system, set `SRI_SEL` to **x** where SR_IN**x** has the highest sampling rate. See Section 5.3, “Multi-Rate Processing” on page 16 for more information about multiple sampling rates.

The CFX can also directly control Filter Engine program execution. You can find more information in the *Hardware Reference Manual for Ezairo 7100*.

5.2 Conditional Processing

Conditional processing is implemented with the use of state codes. All Filter Engine instructions are optionally conditional. The mechanism consists of an optional 4-bit condition in the instruction word, `SC`**x** (where **x** = 0, 1, 2, ... 15), and a 16-bit state code register (`D_FENG_STATE_ACTIVE`) that can be updated either by the CFX, or as a result of a compare (`CMP`) instruction. For example, this instruction executes only if bit 0 of the state code register `D_FENG_STATE_ACTIVE` is set:

```
add [sc0], e5, e6, e7
```

If the conditional operand `SC`**x** is specified but the state code bit is cleared, the instruction is not executed. The Filter Engine idles for the same number of cycles that the instruction would take to execute.

If the conditional operand `SC`**x** is not specified, this instruction is always executed regardless of the contents of the state code register `D_FENG_STATE_ACTIVE`.

5.2.1 Active and Shadow Registers for Conditional Processing

Filter Engine conditional instruction execution is determined by the specified state code in the register `D_FENG_STATE_ACTIVE`. The Filter Engine also writes the output of the `CMP` instruction to this register. It has a shadow register, `D_FENG_STATE_SHADOW`, which the CFX can use to safely update the state code(s). When the appropriate write-enable bits in the `D_FENG_STATE_W_EN` register are set, and the CFX writes to the shadow register `D_FENG_STATE_SHADOW`, its content is transferred to the active register as soon as the Filter Engine becomes idle. If the Filter Engine is continuously active, the transfer occurs at the start of the next program execution.

Alternatively, when the appropriate write-enable bits in the D_FENG_STATE_W_EN register are set, the CFX can write to the active register for the new value to take effect immediately. However, use this feature with caution to avoid unintended memory access conflicts. If a memory conflict occurs, the Filter Engine access to the active register takes precedence over the CFX's access. Any data read by the CFX is invalid and any data written by the CFX is lost.

5.3 MULTI-RATE PROCESSING

The Filter Engine supports multi-rate signal processing through two programmable 4-bit multi-rate counters MRCNT0 and MRCNT1, program sector enabling (register D_FENG_PGM_SECTOR_EN), and program phase memory. For convenience, the following concepts are used:

root sector — The main program where all instructions are executed each time the Filter Engine processing is triggered. Therefore, no *program phase* is associated with it. Each program allocation must hold a root sector that corresponds to the highest sampling rate f_s of the signal path.

program sector — The program sector refers to a segment in the program memory where instructions are grouped by the program phase. The program allocation can also hold any CNT**n**PH**m** (n = 0, 1; m = 0, 1, ..., 15) sectors, where instructions are only executed at program phase **m** associated with the counter **n**.

program phase — At a sampling rate f_{sn}*(n=1,2)* which is lower than the highest sampling rate of the system f_s, the program phase *m* refers to the execution time within the sampling period $T_n=1/f_{sn}$ at a program execution phase of $PH_m = m \cdot \frac{1}{f_s}$. This is the program execution time offset relative to the start of the root sector execution.

Each time the Filter Engine starts processing, it starts executing instructions in the root sector. If one or both multi-rate counters are active, the associated counter value is incremented until the maximum count, the "Division Ratio" (DR), is reached. At that time the counter value resets to 0, and the Filter Engine processes the program sector starting from phase 0. In this way, the Filter Engine can support up to three different sampling rates:

$$f_s,\ \frac{f_s}{MRCNT0_DR + 1},\ \text{and}\ \frac{f_s}{MRCN1_DR + 1}$$

where f_s is the sampling rate of the sample ready signal (see Section 5.1, "Starting Program Execution" on page 15).

5.3.1 Multi-Rate Counters

To enable multi-rate processing, you must configure either or both of the two multi-rate counters. Set the division ratio for the counters to achieve your desired sampling rate as calculated above. Refer to the *Hardware Reference Manual for Ezairo 7100* for information about the D_FENG_MRCNT register, and refer to the *Firmware Reference Manual for Ezairo 7100* to use firmware macros to configure the Filter Engine multi-rate counters.

5.3.2 Phase Memory

While the root sector of the Filter Engine program code cannot be empty and is always executed at the highest sampling rate, you can add program sectors CNT**n**PH**m** where instructions are executed at program phase **m** at the divided sampling rate.

When you add a CNT**n**PH**m** sector in the Filter Engine source code, the corresponding sector is enabled in the Filter Engine program sector enable register D_FENG_PGM_SECTOR_EN. The Filter Engine has a program phase memory consisting of eight 24-bit words, accessible from the CFX as registers D_FENG_PHM_PH0_PH1 to D_FENG_PHM_PH14_PH15. They contain the program memory starting addresses for all enabled phases for the two multi-rate counters. Refer to the *Hardware Reference Manual for Ezairo 7100* for more information about these registers. Initialization data of these registers are generated automatically by the compilation tools; therefore you do not have to configure them manually.

5.4 Numerical Precision and Data Conversion

The following data formats are utilized in the source or destination registers of Filter Engine instructions:

- 24-bit, single-precision frac24 format, data in range [-1, 1). This format is used for input/output registers, intermediate audio samples, as well as FIR filter coefficients. It is supported by all instructions that involve writing a numerical result to a 24-bit destination register.
- 24-bit, single-precision m2p22 format, biquad filter coefficients in range [-2, 2). If you use the BIQ instruction, you are expected to provide biquad coefficients in this format from the CFX.
- 50-bit, extended-precision m4p46 linear format, data in range [-8, 8), used for the extended-precision registers. This format is used to save and accumulate full-precision multiplier or filter-core outputs.
- 24-bit log2 m8p16 format, data in range [-128, 128). This is the output format of the LOG instruction, and the expected format of the first operand in the LMUL, LMLA, and LMLS Instructions. Conversion to this number format is not automatic. When data is converted into this format, it cannot be converted back to linear.
- 50-bit extended-precision m11p39 format, log-format data in range [-1024, 1024). This format is used when log-format data is stored in extended-precision registers. The 24-bit log-format is extended to 50 bits by 3-bit sign extension and 23-bit zero padding.

In general, in the Filter Engine, data conversion from a larger to smaller size occurs with rounding to infinity and saturation. Conversion from a smaller to larger size is achieved with sign extension and zero padding.

5.5 Program Format and Instruction Set

This section first describes the general program format and general instruction format. When each instruction is described in detail, the operands are provided in a shortened form for brevity. The operands are explained in Table 1 on page 18.

5.5.1 Program Format

Format Filter Engine source code in sections as follows:

```
root:
            <start of main program, which always executes>
CNTnPHm:
            <phase memory section start>
CNTnPHm:
            <new phase memory section start>
```

The root section with the label root is the mandatory first section in the Filter Engine source code. Use a NOP instruction if no functional instruction is needed in the root section. The root section always executes.

The root section can be followed by up to 32 phase memory sections defined with labels in the form CNT**n**PH**m** where **n** = 0, 1 and **m** = 0, 1, ..., 15.root and CNT**n**PH**m** are the only labels accepted. Every label can be defined only once

and must occur in the code in the following order (though not all need to be defined, in which case the missing sector would not be activated in the D_FENG_PGM_SECTOR_EN register):

1. CNT0PH**y** where **y** = 0 to 15 in ascending order
2. CNT1PH**y** where **y** = 0 to 15 in ascending order

Defined sections with no instructions are deactivated with a warning issued to standard error.

5.5.2 General Instruction Format

All instructions other than the NOP instruction use the following format:

```
INS [optional codes] dst, src, other arguments
```

where:

INS — instruction; all instructions are described in alphabetic order from “ABS” on page 20 to “SUB” on page 38.

optional codes — operands that you can choose to include or not. Valid optional codes are identified in the “Syntax” section of each instruction description, and defined in Table 1 below.

dst — destination register (see Table 1)

src — source register (see Table 1)

other arguments — additional operands required by some instructions (see Table 1)

All arguments are case-insensitive. Acceptable argument delimiters include spaces, commas and tabs. Specific instruction formats are shown in the following sections.

5.5.3 Instruction Descriptions

The operands for each instruction are explained in Table 1.

Table 1. Operand Definition

ID	Operand	Description	Variable Range
sc	SC**x**	State code for conditional processing If this operand is specified, the ce field of the opcode is 1. If this operand is absent, the ce field of the opcode is 0.	**x** = 0-15
dst-sc	SC**x**	Destination state code (as the output of CMP instruction)	**x** = 0-15
sh	LEFT**x**, RIGHT**y**	Shift value	**x** = 1-3, **y** = 1-4
dst-reg src-reg-1 src-reg-2	E**w** S**x** IN**y** OUT**z**	Data register	**w** = 0-15 **x** = 0-27 **y** = 0-11 **z** = 0-7
c	C**x**	Coefficient set	**x** = 0-31
sv	SV**x**	State variable set	**x** = 0-31

Table 1. Operand Definition (Continued)

ID	Operand	Description	Variable Range
qf	RND, ESS	Quantizer format	RND = rounding ESS = Error spectral shaping
cond	EQ, NE, GT, LT, GTE, LTE, ZERO, NOT_ZERO	Condition codes	EQ = (src1==src2) NE = (src1!=src2) GT = (src1 > src2) LT = (src1<src2) GTE = (src1≥src2) LTE = (src1<=src2) ZERO = (src1==0) NOT_ZERO = (src1!=0)
int	NI, CI, SI, TI	Interrupt control settings	NI = no interrupt CI = clear interrupt SI = set interrupt TI = transition interrupt
atc, rtc	Integer constant **x**	attack or release time constants expressed as bit shift by **x**+2	**x** = 0-15
ad	NM, SM	Absolute value disable (to optionally use the level detector as a smoothing filter)	SM = Smoothing filter enable NM = Normal level detector

ABS

Calculate absolute value

Syntax `ABS [sc, sh] dst-reg, src-reg`

Operation `dst-reg = |src-reg|*2^sh`

Description Calculate the absolute value of the source register and store the result in the destination register with an optional shift.

Execution Time 1 cycle

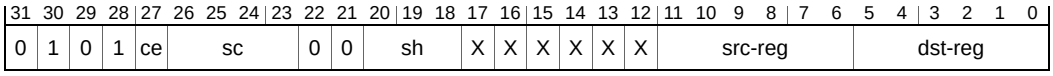
Example 1 `abs e10, e5`

Explanation Take the absolute value of extended-precision data register 5 and store it in extended-precision data register 10 without shifting the result. Since both the source and destination registers have the same number format, no format conversion is triggered.

Example 2 `abs [sc5, right2] s4, e3`

Explanation If state code bit 5 is set, take the absolute value of extended-precision data register 3, shift the result to the right by 2 bits, and store it in single-precision data register 4. Since the destination register precision is less than that of the source register, the shifted result is first rounded and saturated (if necessary) before being stored in single-precision data register 4.

Opcode



ADD

Add registers

Syntax `ADD [sc, sh] dst-reg, src-reg-1, src-reg-2`

Operation `dst-reg = (src-reg-1 + src-reg-2)*2^sh`

Description Add source register 1 to source register 2 and store the result in the destination register with an optional shift. Rounding saturation is based on the width of the destination register.

Execution Time 1 cycle

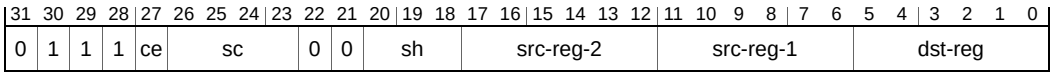
Example 1 `add e4, e2, e3`

Explanation Add extended-precision data register 2 to extended-precision data register 3 and store the sum in extended-precision data register 4. Since both source registers and the destination register have the same number format, no format conversion is triggered.

Example 2 `add [sc1, left1] s2, e2, in2`

Explanation If state code bit 1 is set, add the contents of extended-precision data register 2 to the contents of input register 2, shift the result to the left by 1 bit, and store the result in single-precision data register 2. Since the source register 1 precision is greater than that of the destination register, the shifted result is first rounded and saturated (if necessary) before being stored in register `s2`.

Opcode



BIQ

Perform biquad filtering

Syntax `BIQ [sc] dst-reg, src-reg, c, sv, qf`

Operation `y[n] = c0*x[n] + c1*x[n-1] + c2*x[n-2] + c3*y[n-1] + c4*y[n-2]`

where:

- *x[n]* is the most recent input from src-reg
- *y[n]* is the most recent output stored in dst-reg
- *x[n-1]*, *x[n-2]*, *y[n-1]* and *y[n-2]* are past input and output data maintained by the state variable set `sv`

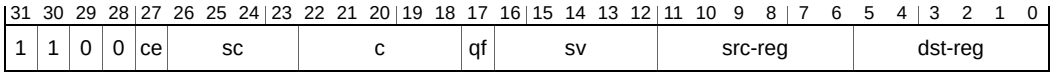
Description Perform one stage of biquad filtering on the specified source register using the specified coefficient and state variable sets, and store the result in the destination register. The filter output is rounded using the rounding mode specified in the `qf` field: either normal rounding (RND) or error spectrum shaping (ESS). After filtering, update the state variable registers with the appropriate data.

Execution Time 2 cycles

Example 1 `biq e7, e9, c3, sv7, rnd`

Explanation Biquad IIR filter using coefficient set 3, state variable set 7, extended-precision data register 9 as the source, and extended-precision data register 7 as the destination, with rounding.

Opcode



CMP

Compare one register with zero or another register

Syntax

```
CMP [sc] dst-sc, src-reg-1, src-reg-2 (or 0), cond, int
```

Operation

```
if (cond is true)
    dst-sc = 1
else
    dst-sc = 0
```

Description

Compare source register 1 with source register 2, and set the destination bit in the state code register if the specified condition is true. If the condition code is `ZERO` or `NOT_ZERO`, only one source register is needed—the second source register is ignored and you can use 0 in its place. The `CMP` interrupt status is updated according to the `int` operand:

- NI: no interrupt
- SI: set an interrupt if the state code register bit changes from 0 to 1
- CI: set an interrupt if the state code register bit changes from 1 to 0
- TI: set an interrupt if the state code register bit changes, either from 0 to 1 or from 1 to 0

For more information about interrupts, see the *Hardware Reference Manual for Ezairo 7100*.

Execution Time 1 cycle

Example 1

```
cmp sc0, e0, e1, gt, si
```

Explanation

Compare extended-precision data register 0 with extended-precision data register 1. If `e0` is greater than `e1`, set bit 0 of the state code register; otherwise, clear this bit. If state code register bit 0 changes from 0 to 1, set the `CMP` interrupt status bit.

Example 2

```
cmp [sc3] sc3, e0, e1, gte, ci
```

Explanation

If state code register bit 3 is set, then compare extended-precision data register 0 with extended-precision data register 1. If `e0` is greater than or equal to `e1`, set bit 3 of the state code register; otherwise, clear this bit.

Note that this code can be used to implement a self-terminating process inside the filter engine. The process is initiated when the CFX sets bit 3 of the state code register. Following this initial event, the filter engine code continues to run as long as the specified condition is met. Otherwise, the `CMP` instruction terminates the process, including all instructions that depend on state code register bit 3.

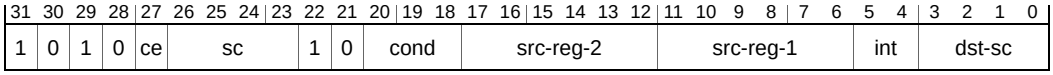
Example 3

```
cmp sc2, e4, 0, zero, ni
```

Explanation

Compare extended-precision data register 4 with zero. If `e4` equals 0, set bit 2 of the state code register; otherwise, clear this bit. No interrupt status is set regardless of the comparison result. The program decoder ignores the `src-reg-2` field.

Opcode


31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
1 0 1 0 ce sc 1 0 cond src-reg-2 src-reg-1 int dst-sc

FIR

Perform FIR filtering

Syntax

```
FIR [sc] dst-reg, src-reg, c, sv
```

Operation

```
if [first stage of FIR]
    dst-reg = c0*x[n] + c1*x[n-1] + c2*x[n-2] + c3*x[n-3] + c4*x[n-4]
```

where:

- *x[n]* is the most recent input in src-reg
- *x[n-1], ..., x[n-4]* are past inputs maintained by the state variable set `sv`.

```
else
    dst-reg = src-reg + c0*sv0 + c1*sv1 + c2*sv2 + c3*sv3 + c4*sv4
```

where `src-reg` is the partial sum from the previous stage and must be the same register as the destination register from the previous stage.

Description

Perform one stage of FIR filtering using the specified coefficient and state variable sets. The initial value of the FIR filter is determined by the source register. If the source register is a single-precision register, this instruction is treated as the first FIR stage of a new filter, and the first FIR state variable is loaded with `src-reg`. The filter core accumulator is cleared to 0 prior to the FIR calculation. If the source register is an extended-precision register (`e0` to `e15`), this instruction is treated as a continuing FIR filter stage, and the filter core accumulator is initialized with the specified extended-precision source register `src-reg`.

Similarly, if the destination register is a single-precision register, this instruction is treated as the final FIR stage. The filter output is rounded and saturated to 24 bits and transferred to the destination register. If the destination register is an extended-precision register (`e0` to `e15`), this instruction is treated as a continuing FIR filter stage, and the filter core accumulator content is transferred to the specified extended-precision destination register without rounding but with 1 bit saturation (51 to 50 bits). The accumulator data is stored in the destination register and is used to initialize the following FIR stage.

Multi-stage FIR filters must be executed in the natural order of the coefficients, and all stages must use consecutive state variable (`sv`) sets. The coefficient set numbers do not have to be consecutive.

Execution Time

2 cycles

Example 1

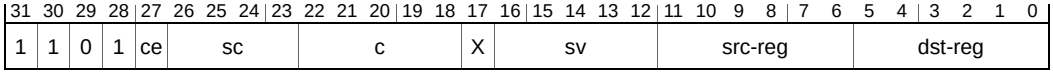
```
// First FIR stage using coefficient set 0 and state variable set 0
fir e0, s2, c0, sv0
// Second FIR stage using coefficient set 1 and state variable set 1
fir e0, e0, c1, sv1
// Third/Final FIR stage using coefficient set 2 and state variable set 2
fir s15, e0, c2, sv2
```

Explanation

These instructions implement a 15th-order FIR filter. The input to the FIR filter is single-precision data register s2. The filter core accumulator is initialized to zero prior to the first FIR stage. The first stage uses coefficient set 0 and state variable set 0 and the output is stored in the extended-precision register e0. The second stage is initialized with register `e0`

and the FIR calculation uses coefficient set 1 and state variable set 1. The output is placed in `e0` with no rounding. The final FIR stage uses coefficient set 2 and state variable set 2 and stores the filter output in single-precision data register s15. The filter output is rounded and saturated to 24 bits.

Opcode



LDET

Detect the level of a signal

Syntax

```
LDET [sc] dst-reg, src-reg, atc, rtc, ad
```

Operation

```
if (ad == 0)
    dst-reg = 2^-(tc+2)*|src-reg| + (1 - 2^-(tc+2))*dst-reg
```

where:

- tc = atc if (|src-reg| - dst-reg) ≥ 0
- tc = rtc if (|src-reg| - dst-reg) < 0

```
else
     dst-reg = 2^-(tc+2)*src-reg + (1 - 2^-(tc+2))*dst-reg
```

where:

- tc = atc if (src-reg - dst-reg) ≥ 0
- tc = rtc if (src-reg - dst-reg) < 0

Description

Perform a level detector operation on the data contained in the source register using the time constants specified, and store the smoothed result in the destination register.

The level detector consists of an absolute value operation (that can be optionally disabled by the `ad` operand) followed by a smoothing filter. The smoothing filter is a dual time constant first-order recursive filter. A constant value of 2 is always added to the specified value of a time constant, so the powers of two actually range from 2 to 17. The smoothing operation can be applied to either single-precision or extended-precision data; however, the slowest time constants can only be achieved for an extended-precision destination register.

Execution Time

2 cycles

Example 1

```
ldet s15, s0, 4, 6, nm
```

Explanation

This example shows the `LDET` instruction used to implement a simple envelope detector. The instruction takes the absolute value of single-precision data register s0 and applies this to the smoothing filter with an attack coefficient of 2^{-6} and a release coefficient of 2^{-8}. Note that in this example, any signal below 2^{-17} will not be detected.

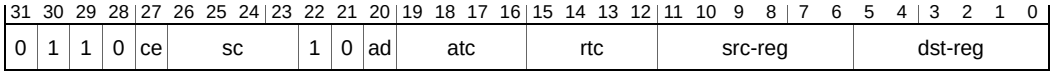
Example 2

```
ldet e1, s0, 14, 14, sm
```

Explanation

This example shows the `LDET` instruction used to implement a smoothing filter (the absolute value operation is disabled) with longer time constants. The output of this `LDET` instruction can be either positive or negative.

Opcode


31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0 1 1 0 ce sc 1 0 ad atc rtc src-reg dst-reg

LMLA

Multiply one register by the inverse log2 of the other register, and add the product to the destination register

Syntax `LMLA [sc, sh] dst-reg, src-reg-1, src-reg-2`

Operation $dst\text{-}reg = (src\text{-}reg\text{-}2 * 2^{src\text{-}reg\text{-}1} + dst\text{-}reg) * 2^{sh}$

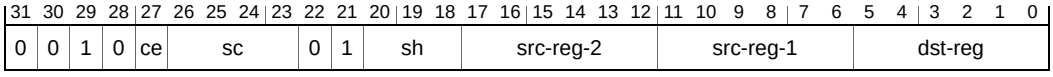
Description Multiply source register 2 by the inverse log2 of source register 1, add it to the destination register and store the result in linear format in the destination register with an optional shift. Any extended-precision source operands are first rounded and saturated before multiplication. The data stored in source register 1 must be in log2 format. Prior to multiplication, the log2 format value is limited to m5p16 with the range of +96 to -96 dB gain (16-bit shift). If the data stored in source register 2 is in extended precision, it is rounded to m4p23 format. The summation with the destination register is in full extended precision. The result is rounded and saturated as required by the destination register.

Execution Time 2 cycles

Example 1 `lmla s4, s1, s17`

Explanation Multiply single-precision data register s17 by the inverse log2 of single-precision data register s1, add the product to single-precision data register s4, apply rounding, apply saturation and store the result in single-precision data register s4.

Opcode



LMLS

Multiply one register by the inverse log2 of the other register, and subtract the product from the destination register

Syntax `LMLS [sc, sh] dst-reg, src-reg-1, src-reg-2`

Operation $dst\text{-}reg = (dst\text{-}reg - src\text{-}reg\text{-}2 * 2^{src\text{-}reg\text{-}1}) * 2^{sh}$

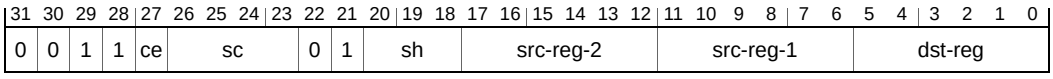
Description Multiply source register 2 by the inverse log2 of source register 1, subtract the product from the destination register and store the result in linear format in the destination register with an optional shift. The data stored in source register 1 must be in log2 format. Prior to multiplication, the log2 format value is limited to m5p16 with the range of +96 to -96 dB gain (16-bit shift). If the data stored in source register 2 is in extended-precision, it is rounded to m4p23 format. The subtraction from the destination register is in full extended precision. The result is rounded and saturated as required by the destination register.

Execution Time 2 cycles

Example 1 `lmls s4, s1, s17`

Explanation Multiply single-precision data register s17 by the inverse log2 of single-precision data register s1, subtract the product from single-precision data register s4, apply rounding, apply saturation, and store the result in single-precision data register s4.

Opcode



LMUL

Multiply one register with the inverse log2 of the other register

Syntax `LMUL [sc, sh] dst-reg, src-reg-1, src-reg-2`

Operation $dst\text{-}reg = src\text{-}reg\text{-}2 * 2^{src\text{-}reg\text{-}1} * 2^{sh}$

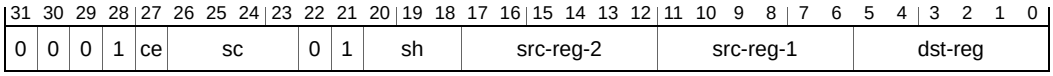
Description Multiply source register 2 by the inverse log2 of source register 1 and store the result in linear format in the destination register with an optional shift. Any extended-precision source operands are first rounded and saturated before multiplication. The data stored in source register 1 must be in log2 format. Prior to multiplication, the log2 format value is limited to m5p16 with the gain range of +96 to -96 dB (16-bit shift). If the data stored in source register 2 is extended-precision, it is rounded to m4p23 format. The result is rounded and saturated as required by the destination register.

Execution Time 2 cycles

Example 1 `lmul s4, s1, s17`

Explanation Multiply single-precision data register s17 by the inverse log2 of single-precision data register s1, apply rounding, apply saturation and store the result in single-precision data register s4.

Opcode



LOG

Calculate base-2 logarithm

Syntax `LOG [sc] dst-reg, src-reg`

Operation `dst-reg = log2(src-reg)`

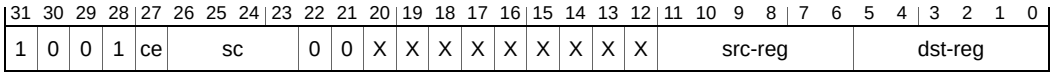
Description Calculate the base-2 logarithm of data in the source register and store the result in the destination register. Source data can be in single-precision linear format (frac24) or extended-precision linear format (m4p46). The result will be in log format (m8p16). The `LOG` function is implemented using a combination of a look-up table and linear interpolation to achieve an overall accuracy that is better than 0.005 dB. This accuracy only requires 12 fractional log2 bits. Therefore in log2 conversion, the 4 LSBs of the single-precision register are always returned as zeros.

Execution Time 1 cycle

Example 1 `log [sc5] s1, e3`

Explanation If state code bit 5 is set, calculate the base-2 logarithm of extended-precision data register e3, and store the result in single-precision data register s1.

Opcode



MLA

Multiply registers and add to the destination register

Syntax `MLA [sc, sh] dst-reg, src-reg-1, src-reg-2`

Operation `dst-reg = (src-reg-1 * src-reg-2 + dst-reg)*2^sh`

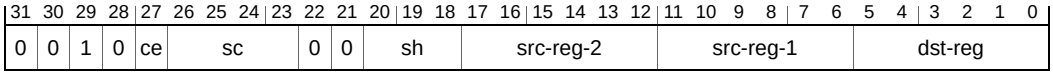
Description Multiply source register 1 with source register 2, add it to the destination register and store the result in the destination register with an optional shift. The multiplier is capable of multiplying 24 bits of source register 1 by the 27 MSBs of source register 2 if it is in extended precision, or by the 24 bits of source register 2 if it is in single precision. If an extended-precision register is specified for source 1, it is rounded and saturated to m1p23 format prior to multiplication. If an extended-precision register is specified for source 2, it is rounded to m4p23 format prior to multiplication. The result is rounded and saturated as required by the destination register.

Execution Time 2 cycles

Example 1 `mla [sc15, left1] e2, s1, e2`

Explanation If state code register bit 15 is set, round extended-precision data register e2 into m4p23 format and multiply with single-precision data register s1 (frac24). Add the result to extended-precision data register e2, and save the result back to extended-precision data register e2 after shifting left by one bit and saturating to m4p46 format.

Opcode



MLS

Multiply registers and subtract from the destination register

Syntax `MLS [sc, sh] dst-reg, src-reg-1, src-reg-2`

Operation $dst\text{-}reg = (dst\text{-}reg - src\text{-}reg\text{-}1 * src\text{-}reg\text{-}2) * 2^{sh}$

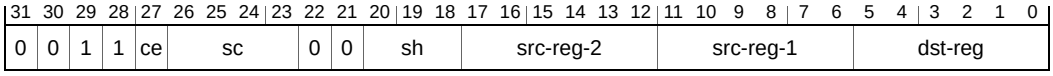
Description Multiply source register 1 with source register 2, subtract the product from the destination register and store the result in the destination register with an optional shift. The multiplier is capable of multiplying 24 bits of source register 1 by the 27 MSBs of source register 2 if it is in extended precision, or by the 24 bits of source register 2 if it is in single precision. If an extended-precision register is specified for source register 1, it is rounded and saturated to m1p23 format prior to multiplication. If an extended-precision register is specified for source register 2, it is rounded to m4p23 format prior to multiplication. The result is rounded and saturated as required by the destination register.

Execution Time 2 cycles

Example 1 `mls [sc6, left1] e0, in2, e0`

Explanation If state code register bit 6 is set, round extended-precision data register e0 into m4p23 format and multiply it with input register in2. Subtract the result from extended-precision data register e0 and save the result back to register e0 after shifting left by one bit, and rounding and saturating to m4p46 format.

Opcode



MOV

Move data from source register to destination register

Syntax `MOV [sc, sh] dst-reg, src-reg`

Operation `dst-reg = src-reg * 2^sh`

Description Move data from the source register to the destination register with an optional shift. The shifted result is rounded and saturated as required by the precision of the destination register.

Execution Time 1 cycle

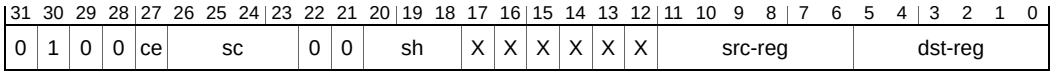
Example 1 `mov s3, s9`

Explanation Move single-precision data register s9 to single-precision data register s3. Since both registers are of the same precision, no format conversion is applied.

Example 2 `mov [left2] s3, e5`

Explanation Shift extended-precision data register e5 to the left by 2 bits, apply 24-bit rounding and saturation, and transfer the result to single-precision data register s3.

Opcode



MUL

Multiply registers

Syntax `MUL [sc, sh] dst-reg, src-reg-1, src-reg-2`

Operation `dst-reg = src-reg-1 * src-reg-2 * 2^sh`

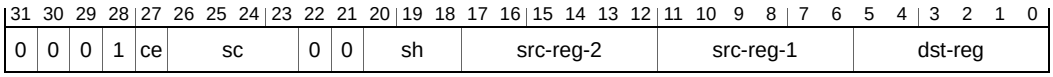
Description Multiply source register 1 by source register 2 and store the result in the destination register with an optional shift. The multiplier is capable of multiplying 24 bits of source register 1 by the 27 MSBs of source register 2 if it is in extended precision, or by the 24 bits of source register 2 if it is in single precision. If an extended-precision register is specified for source register 1, it is rounded and saturated to m1p23 format prior to multiplication. If an extended-precision register is specified for source register 2, it is rounded to m4p23 format prior to multiplication. The result is rounded and saturated as required by the destination register.

Execution Time 2 cycles

Example 1 `mul e3, in1, e2`

Explanation Round extended-precision data register e2 into m4p23 format and multiply with input register in1. Save the result in extended-precision data register e3.

Opcode



NOP

No operation

Syntax `NOP`

Operation `no operation`

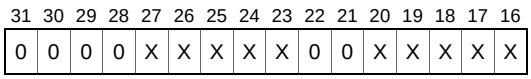
Description Nothing is executed. The decoder increments the program memory counter. Conditional processing fields are ignored.

Execution Time 1 cycle

Example 1 `nop`

Explanation Do nothing.

Opcode



SUB

Subtract registers

Syntax `SUB [sc, sh] dst-reg, src-reg-1, src-reg-2`

Operation `dst-reg = (src-reg-2 - src-reg-1) * 2^sh`

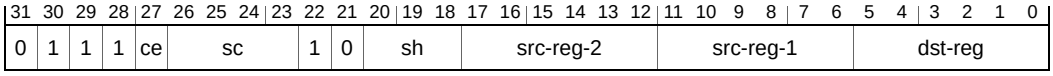
Description Subtract source register 1 from source register 2 and store the result in the destination register with an optional shift. Rounding saturation is based on the width of the destination register.

Execution Time 1 cycle

Example 1 `sub [sc4] e12, e4, e8`

Explanation If the state code register bit 4 is set, subtract extended-precision data register e4 from extended-precision data register e8, and store the result in extended-precision data register e12. Since both source registers and the destination register have the same number format, no format conversion is triggered.

Opcode



31	30	29	28	27	26 25 24	23	22	21 20	19 18 17 16	15 14 13 12	11 10 9 8	7 6 5 4	3 2 1 0
0	1	1	1	ce	sc	1	0	sh	src-reg-2	src-reg-1	dst-reg		

CHAPTER 6
Software Tools

This chapter describes the toolchain and debugging tools available to support the Filter Engine processor.

6.1 Filter Engine Assembler Usage

The Filter Engine Assembler (*feasm*) is a command-line tool that takes an input file containing Filter Engine program code. It outputs a CFX source file for linking with the application. The dataflow of files and tools involving the feasm and the CFX toolchain is depicted in Figure 1. The CFX source is linked into the CFX application by allocating segments of P memory as follows:

- Control and configuration registers: 0xF300 to 0xF312
- State memory: 0xF800 to 0xF8FF
- Coefficient memory: 0xF900 to 0xF9BF
- Program memory: 0xF9C0/0xF9FF

The last three segments are shared memory: 0xF800 to 0xF9FF.

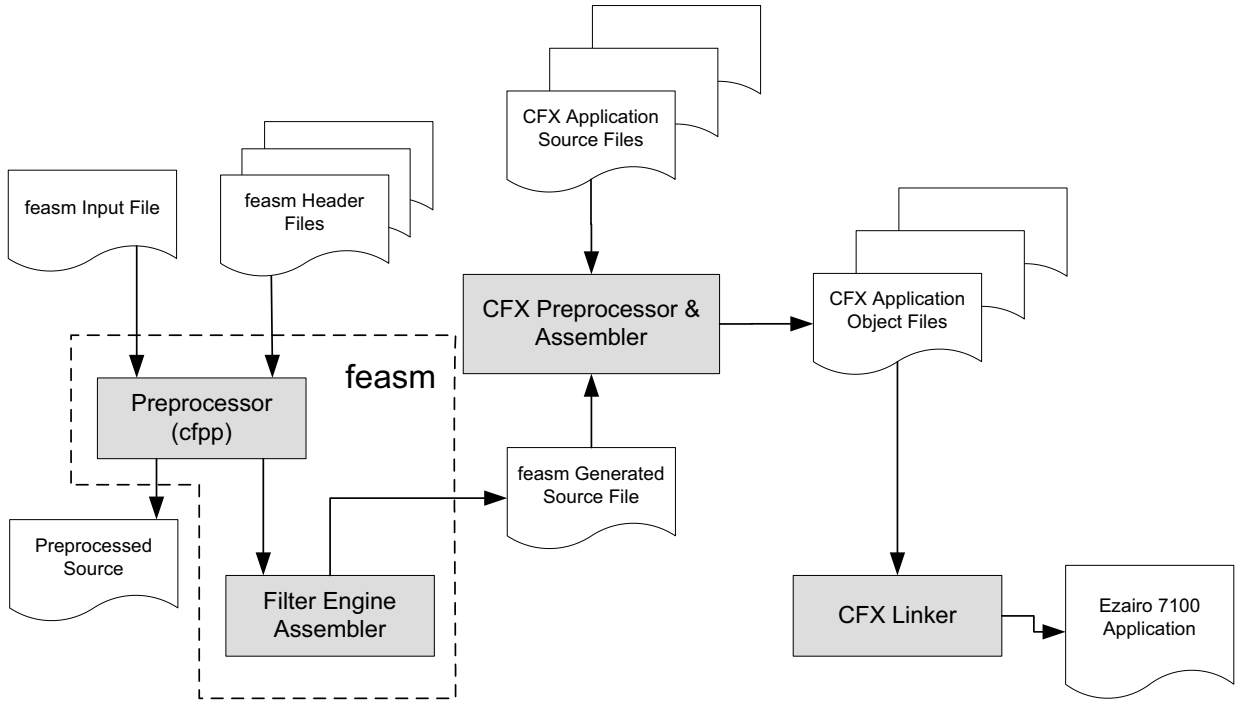


Figure 1. feasm Configuration Usage and Integration with CFX Compilation Toolchain

The *.cfs* file generated by the Filter Engine Assembler contains Filter Engine program code, as well as a Filter Engine program memory sector enable and phase memory configuration. The rest of the Filter Engine configuration and the placement of filter coefficients, if necessary, are done through CFX code.

The syntax for feasm is as follows:

```
feasm inputfile options
```

inputfile is a *.fs* file containing Filter Engine assembly code. **options** can be any one option or a combination of the options listed in Table 1.

Table 1. Filter Engine Assembler Options

Options	Description
/o:**outputfile**	Generate an output file. Specify the output file in the following format: **"path\file"**
/h	Print usage help
/u	Display memory utilization
/w	Show warnings on successful build
/m:**mode**	Specify mode. Default mode: SK4
/ppi:**dirs**	Include paths, semi-colon separated.
/ppd:**defines**	Specify pre-compile defines: **name value** , **name value** , **name value**, …. Delimiter can be comma (,) or semi-colon (;) A white space is used as the delimiter between name and value. A value must be specified.
/ppo:**file**	Generate a preprocessor output file. Specify the output file in the following format: **"path\file"**.

An example of Filter Engine Assembler usage is:

```
feasm /w /ppd:"SR0 32000" /o:feng.cfs feng.fs
```

For information about Filter Engine instructions, see Section 5.5, “Program Format and Instruction Set” on page 17.

6.2 Filter Engine Control and Configuration Through the CFX

Configure the following through the CFX to define the behavior of the Filter Engine:

- Audio multiplexing
- Enable the Filter Engine to capture input samples
- Select which sample starts the Filter Engine processing
- Filter Engine interrupt source
- Enable the Filter Engine master clock; without this, the Filter Engine will not execute.
- If necessary, place filter coefficients into Filter Engine coefficient memory.

For more detailed information, see the *Hardware Reference Manual for Ezairo 7100*.

6.3 The Preprocessor

Prior to compiling your source code, feasm runs it through a C-style preprocessor, the same preprocessor used by CFCC to preprocess CFX assembly code. It supports all of the standard C preprocessor directives, such as `#define`, `#if`, and `#include`, along with several non-standard extensions, such as `#macro` and `#set`.

With one exception, the preprocessor runs exactly as it does in CFCC. The difference is that when the preprocessor is run from feasm, it does not predefine the `_TARGET_<machine>` or `_CFXASM_VER` macros.

For a full description of the preprocessor and all of the directives that it supports, see the *SK4 ELF Toolchain Reference for Ezairo 7100* .

6.4 ASSEMBLER OPERATORS

The operators described in Section 6.4.1, “Arithmetic and Logical Operators”, Section 6.4.2, “Rounding Operators” on page 41, and Section 6.4.3, “Special Operators” on page 42 apply to integer operands only. Hence, they only apply to the `atc` and `rtc` operands as described in Section 5.5.3, “Instruction Descriptions” on page 18.

To indicate that you want to use an expression as the operand that needs to be evaluated by the assembler, enclose the operand in parentheses: “(“ and “)”.

For example:

```
#define SR 32000
#define timems 0.01
#define tc_round  (rnear( log2(SR*timems) ))
#define tc_with_sample_delay(tc_round-2)

LDET [SC0] e3, e0 , tc_round, (tc_round-2), SM
```

6.4.1 Arithmetic and Logical Operators

Directive	Description
`+, -, *, /`	Double-precision (64-bit) addition, subtraction, multiplication, division.
`%`	Modulo (floored division).
`||, &&`	Logical OR, AND. Evaluates to 0 or 1.
`!`	Logical NOT
`^`	Logical XOR
`==, !=`	Equal to, not equal to. Evaluates to 0 or 1.
`>, <`	Greater than, less than. Evaluates to 0 or 1.
`~`	Bitwise inversion
`|, &`	Bitwise OR, AND
`<<, <<`	Left shift, right shift
`(,)`	Statements in parentheses are evaluated first; otherwise, the entire statement is evaluated left to right.

6.4.2 Rounding Operators

Operator	Description
`RNEAR`	Rounding to nearest integer. If the fractional component is halfway between two integers, then the nearest number that is away from zero is returned.
`RCONV`	Convergent rounding (rounding to even).
`FIX`	Truncation (rounding to zero).

Operator	Description
CEL	Ceiling. Returns the smallest integer greater than or equal to the specified floating-point number.
FLR	Flooring. Returns the largest integer less than or equal to the specified floating-point number.

6.4.3 Special Operators

Operator	Description
SIN, COS	Sine, cosine in radians.
LOG2, LOG10, LOGE	Logarithm to the specified base.
POW	Exponent.

6.4.4 Comments and Labels

Operator	Description
//	Line comments.
\	Line continuation.
/* */	Block comments.
label:	Label definition. Legal labels are: ROOT, CNT**n**PH**m** (**n** = 0, 1; **m** = 0, 1, ..., 15)

6.5 Debugging Filter Engine Configurations

The Filter Engine core on the Ezairo 7100 system does not have its own debug port. You can debug a Filter Engine configuration via shared memory (mapped to 0xF800-0xF9FF in P memory) in a CFX debug session as follows:

1. Disable the Filter Engine by modifying the FENG_CTRL register. The CFX debugger does not disable the Filter Engine when it halts the CFX; you must disable it manually. If you fail to disable the Filter Engine before viewing its memory, you will likely cause a memory fault in the Filter Engine.
2. View and modify the Filter Engine memory in the Memory View in the Integrated Development Environment (IDE).

Besides viewing and modifying Filter Engine memory, you can perform more debugging under CFX control with these convenient Filter Engine features: single step, delayed pause, and manual execution. Refer to the Filter Engine chapter in the *Hardware Reference Manual for Ezairo 7100* for more information.

ON Semiconductor and  are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). Ezairo is either a trademark or registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative